

Tool-Agnosticism as Paper 1's Fifth Architectural Claim

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 14, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize Paper 1's fifth named architectural claim — tool-agnosticism at the substrate layer — at paper-claim level, identify the Series A sub-commitments that decompose from it, and state precisely what the claim defends as public prior art.

Abstract

Paper 1 of the Coordination Knowledge Substrate (CKS) theory series ("Coordination Outside the Model," Li, April 2026) defends six architectural commitments. Its fifth named claim — tool-agnosticism — is the substrate-layer accessibility commitment that closes the paper's core-theory defense. The claim's architectural content is precise: any host environment satisfying three minimal requirements (persistent structured state, human read/write access, LLM access to substrate content) can instantiate a compliant CKS substrate, and no host failing any one of the three can. This anchor note formalizes the claim at paper-claim level. It states what each of the three requirements names, states the jointly-sufficient-and-individually-necessary algebraic shape that makes the conjunction independently claimable as well as each requirement claimable on its own, names the link between Claim 5 and Claim 3 (human-governed authority) — tool-agnosticism is what makes governance portable across hosts rather than dependent on a particular vendor's product choices — identifies what the claim defends against, maps the eight derived Series A sub-commitments (A1.05, A2.24–A2.28, A3.24–A3.25), and provides an operational test consisting of three binary checks, one per requirement.

1. The claim at paper-claim level

Paper 1 §7.1 states tool-agnosticism as follows: CKS is a tool-agnostic design pattern; any environment meeting three minimal requirements — persistent structured state, human read/write access, and LLM access to substrate content — can instantiate the pattern. The claim is the fifth of the paper's six named architectural commitments and the one that closes the core-theory defense by stating what the substrate's host must provide, and what it must not be required to provide, for the rest of the architecture to function.

The claim is distinctive against the surrounding 2024–2026 governance-first architecture cluster because comparable architectures require specialized runtimes — AI gateways, governance platforms, agentic hubs — and locate their agnosticism inside those runtimes (multiple model providers, agent protocols, ML backends). CKS's tool-agnosticism is at a different layer: the substrate itself can live in any environment meeting the three requirements, including environments with no specialized runtime at all. The claim's contribution is not that CKS is more agnostic than comparable architectures; it is that CKS's agnosticism operates at the layer underneath specialized runtimes, where the substrate is.

This anchor note formalizes Claim 5 as a named architectural commitment so that the Series A sub-commitments that decompose from it have an explicit paper-level parent. It introduces no content beyond what Paper 1 §7 commits to; the full operational treatment is the work of the foundational note A1.05, with the joint-sufficiency and individual-necessity properties decomposed across A2.24–A2.28 and the failure modes formalized in A3.24–A3.25.

2. The three minimal requirements

The claim has its content in the three named requirements. Each is stated here in the form Paper 1 §7.1 commits to.

Requirement 1 — Persistent structured state. The host environment must support content that persists across reads and writes, with addressable structure the substrate can rely on (entities, relationships, fields, rows, records, or equivalent). Ephemeral storage — per-session context windows, transient caches, in-memory state that does not survive restart — does not satisfy this requirement.

Requirement 2 — Human read/write access. Humans must be able to read and write substrate content directly, without intermediation by an LLM or by a specialized runtime layer as a precondition of access; the interface form is unconstrained (spreadsheet, database client, document editor, wiki, structured-text file).

Requirement 3 — LLM access to substrate content. The LLM used as substrate mediator must be able to read substrate content as input to its operations and write substrate content as output; the access mechanism is unconstrained, but the access must be possible in both directions.

These three are the entirety of the host interface. No fourth requirement is implied, and any further capability a particular host provides may be useful but is not architecturally needed. The host provides the three; the substrate provides everything else — its schema, its content, its orchestration rules, its conflict-handling semantics.

3. Jointly sufficient and individually necessary

Two algebraic properties of the three-requirement conjunction make Claim 5 a precise commitment rather than a marketing-style portability claim.

Joint sufficiency. An environment that satisfies all three requirements can host a CKS substrate. No further environmental capability is required. The host need not be a particular vendor's product, need not run inside a particular runtime, need not expose a particular governance API. The architecture takes the conjunction as enough.

Individual necessity. An environment failing any one of the three cannot host a CKS substrate, regardless of what else it provides. A host with persistent structured state and LLM access but no direct human read/write fails as a substrate host. A host with human read/write and LLM access but no persistence across sessions fails. A host with persistent structured state and direct human access but no LLM access fails. The failure in each case is architectural, not contingent on workflow or deployment.

The conjunction matters for what the claim is defensible against. As prior art, Claim 5 is claimable in two registers simultaneously. In the per-requirement register, each of the three named conditions is independently claimable as a necessary property of a substrate host. In the conjunction register, the three-way combination is independently claimable as the sufficient condition. A system claiming novelty in "we let humans read our memory directly as state" engages individual necessity of Requirement 2; a system claiming novelty in "we provide a complete agent governance environment requiring component X" engages joint sufficiency by adding a fourth requirement the architecture does not have.

A system that satisfies only two of three does not satisfy tool-agnosticism in the CKS sense. The two-of-three configurations are architecturally distinct things, each with its own named failure mode (§5). Naming this explicitly is what makes the claim a precise interface specification — the interface is the conjunction, and partial compliance is not partial hosting.

4. Tool-agnosticism as the portability of human-governed authority

Claim 5 is connected to Claim 3 (human-governed authority — the rights to inspect, modify, and override substrate content and orchestration rules) by a specific architectural relationship: tool-agnosticism is what makes the authority commitment *portable across hosts* rather than contingent on a particular vendor's product choices. Misreading this connection is the central misreading the claim preempts.

Claim 3 commits the architecture to human authority over substrate content as an architectural property, not a procedural promise — humans hold the rights as a property of the system's design, not as a feature a vendor may grant or withdraw. For this to be true, the substrate must be hostable in environments the humans can in fact reach, on terms the humans can rely on, without dependence on a vendor's continued provision of access. If governance depends on a vendor-specific API, runtime, or access path, then the authority Claim 3 commits to is vendor-revocable. A governance commitment that only holds on one vendor's platform is structurally a vendor-revocable promise, not an architectural property.

Tool-agnosticism prevents this collapse. By committing the architecture to three host requirements rather than to a host, Claim 5 lets the substrate live in environments where the humans hold the authority Claim 3 names. Requirement 2 is the operational hinge between the two claims: it is named in Claim 5 as part of the host interface, and it is the precondition under which the inspect/modify/override rights Claim 3 commits to can be exercised at all. Without it, Claim 3 reduces to "human-governed on the platform where the substrate happens to live"; with it, Claim 3 becomes "human-governed wherever a compliant host can be assembled." The first form is a deployment-level promise; the second is an architectural property. The same logic carries to Claims 1, 2, 4, and 6 — tool-agnosticism makes the entire commitment package portable rather than vendor-bound — but the Claim 3 link is load-bearing, because authority that only holds on a single vendor's platform is not architectural authority.

5. What the claim defends against

Stating the claim's defensive scope explicitly is what makes it auditable as prior art. Four classes of system fail Claim 5 in named ways.

Vendor lock-in as governance risk. A substrate whose authority depends on a specific vendor's product features fails the portability the claim commits to. If the vendor changes its API, restricts access, raises its price beyond what the operator can pay, or discontinues the product, governance over the substrate is lost. This is not a deployment risk to be managed by procurement policy; it is a structural failure of Claim 5, and through it of Claim 3. The fix is host-interface compliance — committing the architecture to a host *interface* rather than a host, so the substrate can be moved between any environments that satisfy the interface without architectural reconstruction.

Pure context-window memory as substrate. Holding "substrate" content inside an LLM's context window across turns or sessions fails Requirement 1 (no persistence across sessions in addressable structured form) and fails Requirement 2 (not directly human-readable as state outside model invocation). The failure is in two requirements simultaneously, and either alone disqualifies the configuration. Derived anti-pattern: A3.24.

Black-box agent memory as substrate. Memory held inside an agent framework and accessed only through the agent's API fails Requirement 2: the human cannot read or write the memory directly, only through the agent's mediation. The architectural commitment to direct human access does not survive an API gate, regardless of how permissively the API is designed. Derived anti-pattern: A3.25.

Platform-specific substrate implementations that cannot be migrated. A substrate hosted in an environment that satisfies the three requirements but cannot in principle be migrated to another environment that also does fails the cross-host claim the architecture commits to. A pattern bound to a specific host beyond the three requirements has, in effect, added a fourth requirement and is no longer the kind of host Claim 5 commits to.

A system that uses any of these as a component can still be CKS-coherent if those components are layered over a substrate that does satisfy the three requirements — a vector index built over a human-readable substrate, an agent memory exposed alongside a directly-readable store. The substrate itself must satisfy the requirements; auxiliary components built around or over it need not.

6. Derived sub-commitments mapped

Eight Series A sub-commitments decompose from Claim 5. Each is named here with the work it performs.

- **A1.05 (foundational sub-commitment).** The full operational treatment of the three-requirement host interface: each requirement's content; the jointly-sufficient/individually-necessary properties; the distinction between CKS's tool-agnosticism and adjacent commitments (model-agnosticism, vendor-neutrality, open-source mandates); the operational test for whether a given environment qualifies. A1.05 is the foundational note Claim 5 anchors.
- **A2.24 (operational variant — Requirement 1).** Persistent structured state in full operational form: what "persistent" means across session boundaries and process restarts; what "structured" means as addressable units; the schema/host boundary.
- **A2.25 (operational variant — Requirement 2).** Human read/write access in full operational form: what "direct" means as the absence of LLM or runtime intermediation as a precondition; the distinction between gates beside the substrate (admissible) and gates in front of it (failure).
- **A2.26 (operational variant — Requirement 3).** LLM access to substrate content in full operational form: bidirectional access as the architectural minimum; access-mechanism agnosticism; the read-as-input / write-as-output structure that lets the LLM operate as substrate mediator.
- **A2.27 (operational variant — joint sufficiency).** The conjunction of the three requirements as sufficient: no further capability required; additional capabilities admissible but not preconditions; the commitment that "three is enough."
- **A2.28 (operational variant — individual necessity).** Each requirement as a necessary condition: failure of any one disqualifies a host; named failure modes for each two-of-three configuration.
- **A3.24 (anti-pattern).** Pure context-window memory as substrate: fails Requirements 1 and 2; the named failure mode and its architectural consequences.
- **A3.25 (anti-pattern).** Black-box agent memory as substrate: fails Requirement 2; the named failure mode and the API-gate boundary problem.

Through A0.05, the eight sub-commitments have an explicit named parent at paper-claim level. The decomposition is exhaustive within Series A: every Series A note that turns on tool-agnosticism is one of the eight, and every one of the eight is traceable to Paper 1 §7.1.

7. Operational test

Three binary checks, one per requirement, together verify tool-agnosticism for a given host.

1. Persistence check. Does the host preserve substrate content across reads and writes, in addressable structured form, surviving session boundaries and process restarts? *Required: yes. Otherwise Requirement 1 fails.*

2. Direct access check. Can a human with appropriate access read and write substrate content directly, in inspectable form, without LLM or specialized-runtime intermediation as a precondition of access? *Required: yes. Otherwise Requirement 2 fails.*

3. Mediator access check. Can the LLM used as substrate mediator read substrate content as input to its operations and write substrate content as output, through whatever mechanism the host provides? *Required: yes. Otherwise Requirement 3 fails.*

A host receiving "yes" to all three is a compliant CKS substrate host. A host receiving "no" to any one is not, regardless of what else it provides. The conjunction is the test; no single check is enough, and partial compliance is not partial hosting. The two-of-three configurations are the named failure modes §5 enumerates.

The test makes no commitment about which particular environments in the 2024–2026 infrastructure landscape will pass — that is an empirical question about available tools, not an architectural one. It makes a commitment about what passing means.

8. Why naming this commitment at paper-claim level matters

Tool-agnosticism is the architectural property that prevents the CKS pattern from collapsing into "CKS-on-platform-X." Without Claim 5, every other commitment the paper defends — substrate as coordination artifact, conflict preservation, human-governed authority, AI as substrate mediator, linear-cost scaling — would inherit a hidden dependency on a particular host's continued provision of access. Each commitment would reduce to a procedural promise rather than an architectural property.

Claim 5 closes this. By committing the architecture to a three-requirement host interface, the rest of the architectural commitment package becomes portable: every claim Paper 1 makes is a claim about every compliant host. A CKS substrate hosted in a spreadsheet, a CKS substrate hosted in a structured document store, and a CKS substrate hosted in a relational database are the same architectural object, because they meet the same host-interface specification; they are not three different patterns sharing a name.

This anchor note formalizes Claim 5 at paper-claim level so that the eight Series A sub-commitments that decompose from it have an explicit named parent, and so that subsequent work adopting, extending, composing, or arguing against the CKS pattern can reference Claim 5 as a named architectural commitment rather than as an implicit property of the paper's framing. Work that uses "tool-agnostic" in the sense formalized here — the three-requirement host interface, jointly sufficient and individually necessary — is using the same concept as Paper 1. Work that uses the term to mean something else — agnosticism inside a specialized runtime, vendor-neutrality, model-agnosticism, or any combination of these — is using a different concept, and the difference should be named.

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Tool-Agnosticism as Paper 1's Fifth Architectural Claim*. May 14, 2026. ORCID: 0009-0004-8065-3235.